# IP1 Sky Quest 3S Project Report

Bilingual presentation and teacher questions 中英对照汇报与答辩准备

EN This report explains my IP1 VR game concept, how a player uses it, how I built it in Unity, and how its main C# systems work. The English is written for an IELTS 5.0 level talk. Each English paragraph is followed by a Chinese translation.

中 这份报告解释 IP1 VR 游戏的概念、玩家如何使用、我如何在 Unity 中制作，以及主要 C# 系统如何运行。英文按雅思 5.0 左右的口头汇报难度写成，每一段后面紧跟中文翻译。

## Part One Presentation Script 第一部分 汇报稿

### 1 Project idea 项目概念

EN My IP1 project is Sky Quest, a small world made for Meta Quest 3S. The player visits a sanctuary, meets ancestors, exchanges objects with candles, talks, and flies with a cape. The aim is to make a simple space feel like a journey with movement, memory, and choice.

中 我的 IP1 项目是 Sky Quest，一款面向 Meta Quest 3S 的小型世界。玩家来到圣殿，遇见先祖，用蜡烛兑换物品，进行交流，并穿着斗篷飞行。设计目标是让一个空间通过移动、记忆和选择变成一段旅程。

EN The desktop and Quest scenes share the same idea. The desktop version lets me test with a keyboard and mouse. The Quest version uses head tracking and Touch controllers. I kept the game rules close, so I could compare the two modes when a problem appeared.

中 桌面场景和 Quest 场景使用同一个设计概念。桌面版方便我用键盘鼠标测试；Quest 版使用头显追踪和 Touch 手柄。两种模式的游戏规则尽量一致，这样出现问题时就能互相比较。

### 2 What the player does 玩家流程

EN First, the player enters the sanctuary and sees an English control guide. The guide can close into a narrow tab on the left. The player can walk, look around, and switch between first and third person. In the Quest version, the left stick moves and the right stick turns smoothly.

中 首先，玩家进入圣殿，看到英文操作指南。指南可以收起成左侧窄栏。玩家可以行走、观察四周，并切换第一和第三人称。Quest 版用左摇杆移动、右摇杆平滑转向。

EN Next, the player approaches an ancestor. The exchange tree opens and shows rewards and candle costs. The player can unlock or drag a reward out. A physical pottery or other exchanged object appears in the world. If the pointer stays on an object for three seconds, a short tooltip explains what it does.

中 接着，玩家靠近先祖。兑换树打开，显示奖励和所需蜡烛。玩家可以解锁或把奖励拖出，世界里会出现陶瓷或其他实体兑换物。鼠标或手柄光标停留三秒后，简短提示会说明该物品的用途。

EN The player can pick up, drop, throw, or consume a reward. Some rewards give one unit of flight energy, and others return five candles. Eating is shown with a right hand movement toward the mouth and a crunch sound. The candle counter is shown at the top left of the game screen.

中 玩家可以拿起、放下、投掷或吃掉奖励。有些奖励补充一格飞行能量，另一些返还五根蜡烛。吃物品时，右手会举向嘴边，并播放咀嚼声。蜡烛数量显示在游戏画面的左上角。

EN A separate flying course starts at a blue ring. The player flies through floating flames and follows the next-target hint. The cape energy stars show the remaining power: when one unit is used, one star goes dark. The sign near the course opens detailed flight rules.

中 另有一个从蓝色圆环开始的飞行关卡。玩家飞过悬浮火焰，并跟随下一目标提示。斗篷上的能量星星显示剩余能量：消耗一格时，一颗星星变暗。关卡旁的指示牌可打开详细玩法说明。

## 3 Making the world 场景制作

EN I used the existing IP1 sanctuary as the base. I added the A301 ritual temple pieces and the A427 temple pieces to make an ancient temple area. I placed the main hall, small shrines, gates, columns, statues, steps, paths, and display stands so the route can be understood. I used A242 ceramics for exchange rewards and gave the temple and ceramics materials and textures.

中 我以 IP1 原有圣殿为基础，加入 A301 祭祀古庙和 A427 寺庙模型，形成古寺区域。我安排了主殿、小神龛、门、柱、雕塑、台阶、道路和展示台，使路线易于理解。A242 陶瓷用于兑换奖励；古寺和陶瓷配置了材质与贴图。

EN In Unity, I used scene builder and integration scripts to place many parts in a repeatable way. This is useful because I can rebuild or check both the desktop scene and the Quest scene. The editor scripts also test important scene references, model orientation, and ceramic materials.

中 在 Unity 中，我使用场景构建和整合脚本，以可重复的方式摆放许多部件。这样可重新生成或检查桌面和 Quest 两个场景。编辑器脚本还检查重要场景引用、模型朝向和陶瓷材质。

EN The character has a cape and a walking pose. The walking script changes body and foot positions using time and movement speed, while a support-foot check reduces visible sliding. The cape script creates the cloth shape, stars, and flight effects. These visual scripts do not move the player by themselves.

中 角色具有斗篷和走路姿势。走路脚本根据时间和移动速度改变身体与脚部姿态，并检查支撑脚以减少明显滑步。斗篷脚本生成布面、星星和飞行特效。这些视觉脚本本身不负责移动玩家。

## 4 Software and build settings 软件与构建设置

EN The current Unity project records version 6000.3.20f1. I used C# scripts for game logic, TextMeshPro for readable text, the Unity Input System for controls, and OpenXR for Quest hardware. The Quest build code sets Android, IL2CPP, ARM64, Vulkan, a Meta Quest feature, Touch controller profiles, and single-pass stereo rendering.

中 当前 Unity 工程记录的版本是 6000.3.20f1。我使用 C# 编写游戏逻辑、TextMeshPro 显示清晰文字、Unity Input System 读取操作、OpenXR 连接 Quest 硬件。Quest 构建代码设置 Android、IL2CPP、ARM64、Vulkan、Meta Quest 功能、Touch 手柄配置和单通道双眼渲染。

EN The main Quest scene is Assets/SkyQuest/SkyQuest3S.unity. The desktop scene is Assets/SkyQuest/SkyDesktop.unity. The build script can create an Android APK named SkyQuest3S.apk. A source file describes the build process, but a successful APK still needs a Unity build and a real headset test.

中 Quest 主场景是 Assets/SkyQuest/SkyQuest3S.unity，桌面场景是 Assets/SkyQuest/SkyDesktop.unity。构建脚本可生成名为 SkyQuest3S.apk 的 Android 安装包。源代码说明了构建过程，但 APK 是否成功仍需 Unity 构建和真机测试来确认。

## 5 How the main code works 核心代码原理

EN SkyQuestInput is a small input boundary. It reads Quest triggers, grips, buttons, and sticks. This keeps hardware reading in one place. SkyQuestLocomotion then combines input, camera mode, ground movement, gravity, and flight displacement. The flight controller calculates a movement suggestion, but the locomotion component moves the player once each frame. This helps avoid two scripts pushing the character at the same time.

中 SkyQuestInput 是集中读取输入的小接口，负责 Quest 扳机、握持键、按钮和摇杆。这样手柄数据只在一个位置读取。SkyQuestLocomotion 再把输入、相机模式、地面移动、重力和飞行位移结合起来。飞行控制器计算位移建议，但最终由移动组件在每帧执行一次角色移动，有助于避免两个脚本同时推动角色。

EN CapeEnergySystem owns the energy value. CapeFlightController asks it before a boost. A successful boost uses one unit. The system limits values between zero and the maximum, so a recharge cannot create unlimited energy. CapeEnergyFanHUD and SkyQuestCapeVisual read the same value to draw the fan icon and light the cape stars.

中 CapeEnergySystem 统一保存能量数值。CapeFlightController 在加速前向它请求扣除能量；成功加速消耗一格。能量被限制在零和上限之间，充能不会无限增长。CapeEnergyFanHUD 和 SkyQuestCapeVisual 读取同一个数值，分别绘制扇形图标和点亮斗篷星星。

EN SkyQuestMagicExchange owns the ancestor tree and candle spending. SkyQuestMagicPickup owns a physical reward after it leaves the tree. It handles holding, aiming, dropping, throwing, and eating. When eating finishes, it sends the reward to the correct system: flight energy increases by one or the candle wallet increases by five.

中 SkyQuestMagicExchange 管理先祖树和蜡烛消费。奖励离开先祖树后，由 SkyQuestMagicPickup 管理实体物品，包括持有、瞄准、放下、投掷和食用。食用结束时，奖励会交给正确系统：飞行能量加一，或蜡烛钱包加五。

EN SkyQuestAvatarWalk changes the model pose in LateUpdate, after movement for the frame. It also plays the eating hand gesture. SkyQuestGuideUI draws the foldable guide and the flight sign. SkyQuestItemHoverInfo waits for a steady pointer before it shows a tooltip. These scripts change what the player sees, not the basic game rules.

中 SkyQuestAvatarWalk 在 LateUpdate 中调整模型姿态，也负责吃物品时的手臂动作。SkyQuestGuideUI 绘制可收起的指南和飞行指示牌。SkyQuestItemHoverInfo 在光标稳定停留后显示提示。这些脚本主要改变玩家看到的反馈，不改变基本游戏规则。

EN SkyQuestCommunication draws the chat panel and message bubbles. SkyQuestSpeech can use a linked Meta Wit.ai configuration for voice dictation. The Android code also includes an offline speech path when no linked configuration is available. Voice input depends on microphone permission, the installed speech model or service, and real device testing.

中 SkyQuestCommunication 绘制聊天面板与消息气泡。SkyQuestSpeech 可以使用已连接的 Meta Wit.ai 配置进行语音听写；Android 代码还包含未配置时的离线语音路径。语音功能是否可用取决于麦克风权限、安装的语音模型或服务以及真机测试。

### 6 Development process and checks 制作流程与检查

EN My workflow was: import and prepare scene assets, build the temple area, connect ancestors and rewards, add player movement and flying, add the cape and interface, then test the same actions on desktop and Quest. When a visual or control problem appeared, I checked the responsible script and tried a small change before checking again.

中 我的流程是：导入并整理场景资源，搭建古寺区域，连接先祖和奖励，加入角色移动与飞行，再加入斗篷和界面，最后分别在桌面和 Quest 检查相同行为。出现画面或操作问题时，我先定位负责的脚本，再做小范围修改和复查。

EN The project has editor validation scripts for scene references, the flight flame course, and temple objects. This is helpful, but it does not replace a headset test. I should check the final build in Quest for camera position, comfort, controller input, text size, and frame rate. I should also test voice input on the actual device.

中 工程中有用于检查场景引用、飞行火焰路线和古寺对象的编辑器验证脚本。这些检查有帮助，但不能替代头显测试。最终版本仍应在 Quest 中检查相机位置、舒适度、手柄输入、文字大小和帧率，并在真实设备上测试语音输入。

## Part Two Strict Teacher Questions 第二部分 严格提问与回答

### Why use VR 为什么使用 VR

EN VR lets the player look around, reach toward objects, and feel distance and height while flying. A flat game can show the same information, but it cannot give the same body-based interaction. I still need user testing to show whether this design is comfortable and useful.

中 VR 让玩家能环顾四周、伸手接近物品，并在飞行时感受距离与高度。平面游戏也能展示信息，但身体参与方式不同。我仍需要用户测试来证明设计是否舒适、有用。

### What is the difference between the two scenes 两个场景有什么区别

EN SkyDesktop uses keyboard and mouse, while SkyQuest3S uses headset tracking and Touch controllers. The core ideas, ancestor exchange, rewards, and flight course are shared. Separate scenes make device-specific input and UI easier to test.

中 SkyDesktop 使用键盘鼠标，SkyQuest3S 使用头显追踪和 Touch 手柄。先祖兑换、奖励和飞行关卡等核心内容共享。分开场景便于测试设备专属输入和界面。

### Why does one script move the player 为什么只让一个脚本移动角色

EN If the flight script and the walking script both change the player position in one frame, the character can jump, move twice, or cause camera discomfort. CapeFlightController computes the flight step, and SkyQuestLocomotion applies the final movement.

中 如果飞行脚本和行走脚本同一帧都改玩家位置，角色可能跳动、移动两次或使镜头不适。CapeFlightController 计算飞行位移，SkyQuestLocomotion 执行最终移动。

### How does boost spend energy 加速如何消耗能量

EN The flight controller calls TryConsumeBoostEnergy before a boost. If energy is too low, the boost is rejected. If it succeeds, one energy unit is removed and the cape display updates from the shared value.

中 飞行控制器在加速前调用 TryConsumeBoostEnergy。能量不足时拒绝加速；成功时扣除一格，斗篷显示从共享能量值更新。

### What is a state machine 什么是状态机

EN The flight code uses named states such as Grounded, Takeoff, Ascending, Gliding, Falling, and Landing. Each state tells the code which physics and visual response to use. This is clearer than many unrelated true-or-false flags.

中 飞行代码使用 Grounded、Takeoff、Ascending、Gliding、Falling、Landing 等状态。状态告诉代码当前该使用哪种物理和视觉反馈，比大量互不相关的真假变量更清楚。

### How is the right stick made smooth 右摇杆为什么平滑

EN The code reads the stick, ignores a small dead zone, and changes turn speed gradually. It uses frame time, so movement stays more similar across different frame rates. The player can turn without a sudden full-speed jump.

中 代码读取摇杆，忽略中央附近的小幅抖动，并逐步改变转向速度。它使用每帧时间，使不同帧率下的速度更接近；转向不会突然达到最大值。

### How do you stop accidental jumps while exchanging 如何避免兑换时误跳

EN The exchange menu captures its own pointer input. Movement input should be ignored while the menu or drag action owns that input. The important test is to drag an item out and confirm the player height and camera distance do not change.

中 兑换菜单接管对应的光标输入。菜单或拖拽占用输入时，移动逻辑应忽略它。关键测试是拖出物品后确认玩家高度和相机距离没有变化。

### Why use a physical pottery reward 为什么使用实体陶瓷奖励

EN A242 ceramics make the exchange result visible in the world. The player can inspect and hold a reward instead of seeing only a number in a menu. A separate pickup component handles its physical interaction after exchange.

中 A242 陶瓷让兑换结果在世界里可见。玩家能观察和拿起奖励，而不只是在菜单里看到数字。奖励离开兑换树后由独立的拿取组件管理实体交互。

### How does a tooltip appear after three seconds 三秒提示如何出现

EN The hover script remembers the current pointed item and the time when pointing began. If the target stays the same long enough, the tooltip appears. Changing or leaving the target resets the timer, so random pointer movement does not fill the screen with text.

中 悬停脚本记住当前指向物品和开始指向的时间。目标持续不变足够久时显示说明；目标改变或离开时计时重置，避免光标乱动时满屏都是文字。

### How does eating a reward work 吃物品怎样运行

EN The pickup code checks that a valid reward is held and that the action is allowed. It starts a short consumption animation, calls the walking avatar for the right-hand gesture, plays a crunch effect, then removes the reward and adds the correct energy or candle value.

中 拿取代码先检查手中有有效奖励、操作允许，然后启动短暂的食用动画，调用角色脚本做右手动作、播放咀嚼声，最后移除奖励并增加对应能量或蜡烛。

### Why are models and materials separate 为什么模型和材质分开

EN The model gives a shape, while a material controls surface color and texture. One temple mesh can look wrong if its rotation is wrong or its material is missing. The integration script therefore places models, assigns materials, and validates important objects.

中 模型给出形状，材质控制表面颜色与贴图。寺庙网格如果朝向错误或材质缺失，画面仍会出问题。因此整合脚本负责摆放模型、指定材质并检查关键对象。

### What is the difference between Update and LateUpdate Update 和 LateUpdate 有何区别

EN Update runs during the normal frame and is good for reading input. LateUpdate runs later in the frame, after other movement has happened. The walk pose can then follow the final player position for that frame.

中 Update 在普通帧流程中运行，适合读取输入。LateUpdate 在同一帧稍后运行，其他移动已经发生，因此走路姿态可跟随本帧最终位置。

### Does Meta voice work without setup Meta 语音不用配置吗

EN The Meta path needs a valid linked Wit.ai configuration and service access. The code has an Android offline fallback, but that path needs its model and real-device checks too. I would not claim voice works on every headset until I test a final build.

中 Meta 路径需要有效的 Wit.ai 配置和服务访问。代码还有 Android 离线备用路径，但它同样需要模型和真机检查。最终安装包未测试前，我不会声称每台头显都能使用语音。

### What does the editor build script do 编辑器构建脚本负责什么

EN The editor script creates and validates the two scenes, sets Android and OpenXR settings, then asks Unity to build the APK. It is not a script that runs during normal gameplay on Quest. This separation keeps build tools out of the player app.

中 编辑器脚本创建和验证两个场景，设置 Android 与 OpenXR，再让 Unity 构建 APK。它不是 Quest 游戏中持续运行的脚本。这样构建工具不会进入玩家的正常操作流程。

### What are the limits of this prototype 原型有什么限制

EN The world is a focused student prototype. Scene beauty, hand movement, speech quality, and VR comfort still depend on device performance and user testing. I can explain how the code is designed, but I should show test evidence before saying a feature is perfect.

中 这是聚焦于核心体验的学生原型。场景美观、手部动作、语音质量和 VR 舒适度仍受设备性能与用户测试影响。我可以说明代码的设计，但说某项功能完全可靠前应拿出测试证据。

## Part Three Facts To Check During Demonstration 第三部分 演示时可核对的事实

EN Core scenes: Assets/SkyQuest/SkyQuest3S.unity and Assets/SkyQuest/SkyDesktop.unity. Flight practice scene: Assets/SkyQuest/CapeFlightTest.unity. Main runtime scripts: SkyQuestLocomotion, CapeFlightController, CapeEnergySystem, SkyQuestMagicExchange, SkyQuestMagicPickup, SkyQuestCommunication, and SkyQuestGuideUI.

中 核心场景：Assets/SkyQuest/SkyQuest3S.unity 和 Assets/SkyQuest/SkyDesktop.unity。飞行练习场景：Assets/SkyQuest/CapeFlightTest.unity。主要运行脚本：SkyQuestLocomotion、CapeFlightController、CapeEnergySystem、SkyQuestMagicExchange、SkyQuestMagicPickup、SkyQuestCommunication 和 SkyQuestGuideUI。

EN Desktop controls shown in the project guide: WASD moves, mouse looks, Space boosts, 1 and 3 switch view, F opens a nearby ancestor tree, Z picks up or drops, E eats or equips, and holding Q aims a throw. The Quest guide shows left stick movement, right stick smooth turning, A boost, B view change, right grip pickup, and right trigger throw or eat near the face.

中 工程指南中的桌面操作：WASD 移动、鼠标观察、空格加速、1/3 切视角、F 打开附近先祖树、Z 拿取或放下、E 食用或装备、按住 Q 瞄准投掷。Quest 指南包括左摇杆移动、右摇杆平滑转向、A 加速、B 切视角、右握持键拿取，以及右扳机投掷或靠近嘴边时食用。

EN Build configuration in source: Android, OpenXR, Meta Quest Touch profiles, IL2CPP, ARM64, Vulkan, and single-pass stereo. Current project version file: Unity 6000.3.20f1. These settings describe the project source and should be checked in the final Unity editor before a live demonstration.

中 源代码中的构建设置：Android、OpenXR、Meta Quest Touch 配置、IL2CPP、ARM64、Vulkan 和单通道双眼渲染。当前工程版本文件为 Unity 6000.3.20f1。演示前仍应在最终 Unity 编辑器中核对这些设置。